

Installing and Building DBeaver CE from Source on Windows 11

Prerequisites & Environment: DBeaver CE is an Eclipse-based (RCP/RAP) Java application. Recent versions require **Java 21 or higher** and Maven 3.9+ ¹. First verify your setup: open a Command Prompt and run:

```
java -version
mvn -version
```

Ensure `java -version` shows JDK 17 or above and that your `JAVA_HOME` and `PATH` include the JDK and Maven bin directories. (Note: DBeaver's official docs recommend JDK 21 or later ¹. If you have only JDK 17, the build may still succeed, but you might need to upgrade if you encounter errors.)

Clone Repositories: In a folder of your choice, clone both the DBeaver CE source and its *dbeaver-common* dependency:

```
git clone https://github.com/dbeaver/dbeaver.git
git clone https://github.com/dbeaver/dbeaver-common.git
```

You should end up with a **dbeaver** folder containing a root `pom.xml` and submodules (plugins, bundles, etc.), and a parallel **dbeaver-common** folder. This matches the structure expected by the build (see [22] and [30]).

1. Configure Eclipse IDE

DBeaver is an **Eclipse RCP/RAP** application. Use the "Eclipse IDE for RCP and RAP Developers" (version around 2024-09 is recommended) ². In Eclipse:

- **Install DBeaver update site:** Go to **Help → Install New Software**, and in the "Work with" field enter:

```
https://p2.dev.dbeaver.com/eclipse-repo/
```

Check all plugins that appear (these are DBeaver's required OSGi bundles) and finish the install ³.

- *(Optional)* Also install the Eclipse Color Theme plugin from `https://eclipse-color-theme.github.io/update` if you like.
- **Restart Eclipse** when prompted.

These steps load DBeaver's own plugin repository into Eclipse ³. DBeaver's wiki notes that the **2024-09** Eclipse release is tested; older or much newer versions may have compatibility issues ².

2. Import Projects into Eclipse

With Eclipse restarted and the repositories cloned:

- In Eclipse, go to **File → Import → Maven → Existing Maven Projects** ⁴.
- Select the `dbeaver` folder you cloned. Eclipse will detect all submodules (plugins, features, etc.). Make sure they are checked and click **Finish**.
- Repeat the import for the **dbeaver-common** folder (this contains shared libraries and utilities).

Eclipse will then begin resolving dependencies and configuring projects. It may prompt to install additional extensions (Tycho connectors); if so, accept those installs. The IDE should build all imported projects automatically (or you can trigger a build with **Ctrl+B**) ⁵.

If some projects show errors (red X), do **Right-click → Maven → Update Project...** on those projects and then rebuild. In many cases simply restarting Eclipse also completes the build.

Tip: You might encounter “Access restriction” compiler warnings (due to JDK internals). You can right-click such errors in the Eclipse Problems view, choose **Quick Fix**, and change the severity from Error to Warning to silence them ⁶.

Lifecycle Mapping (Tycho): Because DBeaver uses Maven Tycho (Eclipse plugin packaging), you may need to adjust Eclipse's lifecycle mapping. Before importing, go to **Window → Preferences → Maven → Lifecycle Mapping**, click **Open Workspace...**, switch to **Source**, and paste in the Tycho mapping XML from the DBeaver wiki ⁷. This tells Eclipse's Maven integration how to handle Tycho plugins. (If you skip this, Eclipse may prompt to install the Tycho M2E connectors when importing, which you should allow.)

3. Resolve Dependencies

After import, Eclipse (via M2E/Tycho) will fetch all required libraries. This may take a few minutes. You should see the projects in **Hierarchical Presentation** under a DBeaver working set. Ensure **Project → Build Automatically** is enabled. If prompted to accept any additional plugin installations (Tycho or others), click **Install**. Once complete, the DBeaver projects should compile without errors. If not, updating the Maven projects as above usually fixes unresolved references ⁸.

4. Common Windows Build Tips

- **Java Version:** If you get compilation errors related to Java APIs, double-check that Eclipse is using JDK 17 (or higher). In **Window → Preferences → Java → Installed JREs**, ensure your JDK is selected.
- **Long Paths:** On Windows, enable long file paths (e.g. via Registry or Group Policy) if you hit “Path too long” errors, as Eclipse/Tycho can create deep build paths.
- **Tycho Plugins:** Make sure the Tycho build plugins are recognized. If Eclipse complains about missing Tycho components when importing, either let it install the connectors or apply the lifecycle mapping from above.

- **Git Bash vs CMD:** When running build scripts, use the regular Windows Command Prompt or PowerShell (Git Bash may have issues with the `.cmd` script).

If you encounter build failures, check the console for hints and consider cleaning the workspace (`Project → Clean`) or running a Maven update on projects. For example, some users saw a failure in `DBeaver CE Standalone` when using an older Java ⁹ ; switching to JDK 21/17 typically fixes this.

5. Building via Command Line

You can also build DBeaver completely from the command line without Eclipse. In your cloned `dbeaver` directory, run:

```
cd tools
build.cmd
```

This Maven/Tycho script will compile all modules, assemble features, and package the products ¹⁰ . After it finishes, the assembled binaries (installable .zip or folders) will be placed under `product/community/target/products` ¹¹ . For example, you should see a directory like `Windows_x86_64/dbeaver/` containing `dbeaver.exe` and related files. If the build fails, ensure your `JAVA_HOME` and `M2_HOME` point correctly and that you ran a fresh clone of both repos.

6. Running DBeaver Locally

In Eclipse: To run/debug DBeaver as an Eclipse RCP app, import the provided launch configuration or create one:

- Go to **File → Import → Run/Debug → Launch Configurations** (or use the existing `DBeaver.product.launch` if available).
- Alternatively, open **Run → Debug Configurations**, create a new **Eclipse Application**, and select **DBeaver.product** as the target (this should be auto-populated from the product config) ¹² .
- Click **Debug** (or **Run**). This starts a new Eclipse window running DBeaver. If you see “No application id has been found” or missing plug-in errors, open the **Plug-ins** tab in the run configuration and click **Add Required Plug-ins** to fill in missing dependencies ¹³ . After that, the product should launch.

From the CLI build: If you built via `build.cmd`, simply navigate to `product/community/target/products/Windows_x86_64/dbeaver/` and double-click `dbeaver.exe` (or run it from a console). This is the standalone application packaged by the build ¹¹ . Your workspace and connections will be preserved (from your user’s `DBeaverData` folder) even when running the dev build.

7. Project Structure (Overview)

Understanding DBeaver’s source layout helps in finding code to modify ¹⁴ . Key folders:

- **docs/** – assorted documentation (often outdated).
- **features/** – Eclipse *feature* descriptors (metadata to bundle plugins into the product). No Java code here.
- **bundles/** – fundamental plugin modules (e.g. utilities).
- **plugins/** – the main Java source. Notable packages:

- `org.jkiss.dbeaver.model` – Core data model and APIs (no UI).
- `org.jkiss.dbeaver.core` – Main application logic and many base UI classes.
- `org.jkiss.dbeaver.ui.app.standalone` – RCP application launcher for standalone DBeaver.
- `org.jkiss.dbeaver.ui.app.eclipse` – DBeaver’s integration as an Eclipse plugin (adds menus/views when running in Eclipse IDE).
- `org.jkiss.dbeaver.ext.*` – Extensions, especially database drivers and editors for specific databases.
- **product/** – configuration for building the final products (the `.product` file, branding, etc.).

This modular design separates the **data model** (`*.model`) from the **UI** (`*.core`, `*.ui.*`) and from **extensions** (`*.ext`) ¹⁵.

8. Getting Started Contributing

Once you have the project building locally, pick an area of interest. For example, UI changes go in `org.jkiss.dbeaver.core` or one of the `ui.app.*` plugins, while database-specific features live in the appropriate `org.jkiss.dbeaver.ext.<db>` module. The DBeaver wiki and GitHub issues can guide you: check for “good first issue” labels or look at the [Contribute guide](#) for coding standards and submission procedures ¹⁶. As you make changes, you can run DBeaver from Eclipse (step 6) or rebuild with `build.cmd` to test. Over time, you’ll learn the class structure (for example, UI views/editors vs. data model classes) and best commit practices from the contributor docs. The **DBeaverCE** community is active, so don’t hesitate to ask questions on the issue tracker if you hit a snag.

References: Official DBeaver build and Eclipse instructions ¹ ¹⁷ ⁴ ¹² ¹⁵ were followed to assemble these steps. They provide detailed guidance on prerequisites, Maven/Tycho setup, and project structure. The exact lines cited above point to relevant parts of the DBeaver documentation and wiki.

¹ ¹⁰ ¹¹ [raw.githubusercontent.com](https://raw.githubusercontent.com/dbeaver/dbeaver/wiki/aboutgh/about-pub/Build-from-sources.md)

<https://raw.githubusercontent.com/dbeaver/dbeaver/wiki/aboutgh/about-pub/Build-from-sources.md>

² ³ ⁴ ⁵ ⁶ ⁷ ⁸ ¹² ¹³ ¹⁴ ¹⁵ ¹⁷ [Develop in Eclipse · dbeaver/dbeaver Wiki · GitHub](#)

<https://github.com/dbeaver/dbeaver/wiki/Develop-in-Eclipse>

⁹ [Can't build on Windows 11 · Issue #16297 · dbeaver/dbeaver · GitHub](#)

<https://github.com/dbeaver/dbeaver/issues/16297>

¹⁶ [Contribute your code · dbeaver/dbeaver Wiki · GitHub](#)

<https://github.com/dbeaver/dbeaver/wiki/Contribute-your-code>